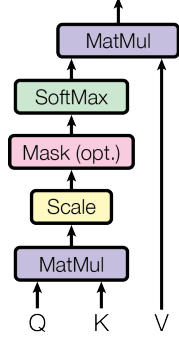


Scaled Dot-Product Attention



Multi-Head Attention

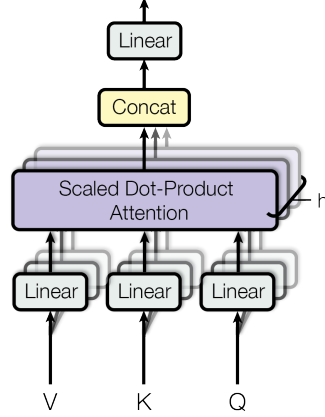


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key.

3.2.1 Scaled Dot-Product Attention

We call our particular attention "Scaled Dot-Product Attention" (Figure 2). The input consists of queries and keys of dimension d_k , and values of dimension d_v . We compute the dot products of the query with all keys, divide each by $\sqrt{d_k}$, and apply a softmax function to obtain the weights on the values.

In practice, we compute the attention function on a set of queries simultaneously, packed together into a matrix Q . The keys and values are also packed together into matrices K and V . We compute the matrix of outputs as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

The two most commonly used attention functions are additive attention [2], and dot-product (multiplicative) attention. Dot-product attention is identical to our algorithm, except for the scaling factor of $\frac{1}{\sqrt{d_k}}$. Additive attention computes the compatibility function using a feed-forward network with a single hidden layer. While the two are similar in theoretical complexity, dot-product attention is much faster and more space-efficient in practice, since it can be implemented using highly optimized matrix multiplication code.

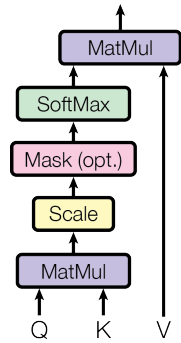
While for small values of d_k the two mechanisms perform similarly, additive attention outperforms dot product attention without scaling for larger values of d_k [3]. We suspect that for large values of d_k , the dot products grow large in magnitude, pushing the softmax function into regions where it has extremely small gradients⁴. To counteract this effect, we scale the dot products by $\frac{1}{\sqrt{d_k}}$.

3.2.2 Multi-Head Attention

Instead of performing a single attention function with d_{model} -dimensional keys, values and queries, we found it beneficial to linearly project the queries, keys and values h times with different, learned linear projections to d_k , d_k and d_v dimensions, respectively. On each of these projected versions of queries, keys and values we then perform the attention function in parallel, yielding d_v -dimensional

⁴To illustrate why the dot products get large, assume that the components of q and k are independent random variables with mean 0 and variance 1. Then their dot product, $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$, has mean 0 and variance d_k .

Scaled Dot-Product Attention



Multi-Head Attention

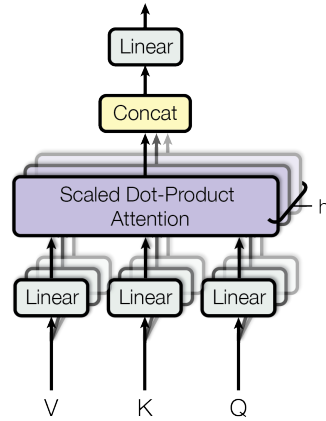


图 2：（左）缩放的点积注意力。（右）多头注意力由多个并行运行的注意力层组成。

值的权重，其中分配给每个值的权重是通过查询与相应键的兼容性函数计算的。

3.2.1 缩放点积注意力

我们将我们的特别注意力称为“缩放点积注意力”（图 2）。输入由维度 d_k 的查询和键以及维度 d_v 的值组成。我们使用所有键计算查询的点积，将每个键除以 $\sqrt{d_k}$ ，然后应用 softmax 函数来获取值的权重。

在实践中，我们同时计算一组查询的注意力函数，并将它们打包成一个矩阵 Q 。键和值也一起打包成矩阵 K 和 V 。我们将输出矩阵计算为：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

两种最常用的注意力函数是加性注意力[2]和点积（乘法）注意力。除了 $\frac{1}{\sqrt{d_k}}$ 的缩放因子之外，点积注意力与我们的算法相同。附加注意力使用具有单个隐藏层的前馈网络来计算兼容性函数。虽然两者在理论复杂性上相似，但点积注意力在实践中更快、更节省空间，因为它可以使用高度优化的矩阵乘法代码来实现。

虽然对于较小的 d_k 值，两种机制的表现相似，但加性注意力优于点积注意力，而无需针对较大的 d_k 值进行缩放 [3]。我们怀疑，对于较大的 d_k 值，点积的幅度会变大，从而将 softmax 函数推入梯度极小的区域⁴。为了抵消这种影响，我们将点积缩放 $\frac{1}{\sqrt{d_k}}$ 。

3.2.2 多头注意力

我们发现，使用不同的学习线性投影分别将查询、键和值线性投影到 d_k 、 d_k 和 d_v 维度 h 次，而不是使用 d_{model} 维度的键、值和查询来执行单个注意力函数。 On each of these projected versions of queries, keys and values we then perform the attention function in parallel, yielding d_v -dimensional

⁴To illustrate why the dot products get large, assume that the components of q and k are independent random variables with mean 0 and variance 1. Then their dot product, $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$, has mean 0 and variance d_k .